

SUPER RESUMEN — TALLER DE PROGRAMACIÓN

Compilado de: `resumen final taller.docx` + `FinalTaller.ipynb`

1. PIPELINE DE COMPILACIÓN

Código fuente → Preprocesado → Compilación → Ensamblado → Enlazado → Ejecutable

Eta ­ pa	En ­ trada	Salida	Qué hace
Pre ­ proce ­ sado	Código fuente	Código pre ­ proce ­ sado	Resuelve <code>#define</code> , <code>#include</code> , <code>#if/#ifdef/#ifndef</code> , elimina comentarios
Com ­ pila ­ ción	Código pre ­ proce ­ sado	Código ensamblador	Analiza sintaxis, verifica tipos
En ­ samblado	Código ensamblador	Código objeto (.o)	Convierte a binario
En ­ lazado	Archivos .o + librerías	Ejecutable	Une todo en un binario final

2. MACROS (`#define`) Y COMPILACIÓN CONDICIONAL

```
#define PI 3.14159
#define CUADRADO(x) ((x) * (x)) // Siempre usar paréntesis en expresiones
```

Buenas prácticas:

- Nombres en MAYÚSCULAS
- Paréntesis en expresiones para evitar precedencia incorrecta
- Sin punto y coma al final
- Preferir `const` o `inline` cuando sea posible (no ocupan memoria)

Compilación condicional (header guards):

```
#ifndef MI_CLASE_H
#define MI_CLASE_H
class MiClase { /* ... */ };
#endif
```

Code bloat: crecimiento excesivo del ejecutable por duplicación de código (ej: instanciar muchos templates con tipos distintos). Se evita factorizando código y usando templates con cuidado.

3. VARIABLES STATIC

Tipo	Scope	Vida	Segmento
Variable global static	Solo ese archivo	Todo el programa	data/BSS
Variable local static	Solo esa función	Todo el programa (persiste entre llamadas)	data/BSS
Atributo de clase static	Todos los objetos de la clase	Todo el programa	data

```
class Persona {
public:
    static int contador;    // pertenece a la clase, no a instancias
};
int Persona::contador = 0; // inicialización fuera de la clase
```

4. PUNTEROS A FUNCIONES Y DECLARACIONES C

```
static int (*f)(short int *, char *); // puntero a función, solo visible
en el módulo
unsigned char *A[5];                  // array de 5 punteros a unsigned
char (visible externamente)
static float *A;                      // puntero a float, solo visible en
el módulo
```

5. ENDIANNESS

Big-Endian: el byte más significativo (MSB) está primero en memoria.

Little-Endian: el byte menos significativo (LSB) está primero.

Conversión Big-Endian (4 bytes) → decimal:

```
uint32_t num = (uint32_t)bytes[0] << 24 |
               (uint32_t)bytes[1] << 16 |
               (uint32_t)bytes[2] << 8 |
               (uint32_t)bytes[3];
// También: num = ntohl(c); // network to host long
```

Conversión decimal → Big-Endian (para escribir):

```
salida[0] = (uint8_t)(num >> 24);
salida[1] = (uint8_t)(num >> 16);
salida[2] = (uint8_t)(num >> 8);
salida[3] = (uint8_t)(num);
```

Base N → decimal:

```
// 3 símbolos base 7 big-endian:
uint32_t val = bytes[0]*49 + bytes[1]*7 + bytes[2];
// General: val = val * N + digito
```

Decimal → Base N (texto):

```
char buf[65]; int i = 0;
if (num == 0) { buf[i++] = '0'; }
while (num > 0) { buf[i++] = (num % N) + '0'; num /= N; }
// buf está al revés → invertir para obtener el resultado
for (int j = 0; j < i; j++) salida[j] = buf[i - j - 1];
// Para bases >10: (resto < 10) ? '0'+resto : 'A'+(resto-10)
```

6. PROCESAMIENTO DE ARCHIVOS EN C (ISO C)

Funciones clave

```
FILE *f = fopen("archivo.dat", "r+b"); // r+ = leer y escribir, b =
binario
fread(&var, sizeof(var), 1, f); // devuelve cantidad leída
fwrite(&var, sizeof(var), 1, f);
fseek(f, offset, SEEK_SET/SEEK_CUR/SEEK_END);
fclose(f);
ftruncate(fileno(f), nuevo_tamaño); // truncar archivo (POSIX)
fputc(0, f); // escribir un byte (para reservar
espacio)
```

Patrón: procesar sobre sí mismo SIN cargar en memoria

Caso "eliminar" o "comprimir" (pos_escritura <= pos_lectura, siempre avanza):

```
long pos_escritura = 0, pos_lectura = 0;
while (1) {
```

```

fseek(f, pos_lectura, SEEK_SET);
if (fread(&dato, sizeof(dato), 1, f) != 1) break;
pos_lectura += sizeof(dato);
if (condicion_para_conservar) {
    fseek(f, pos_escritura, SEEK_SET);
    fwrite(&dato, sizeof(dato), 1, f);
    pos_escritura += sizeof(dato);
}
}
ftruncate(fileno(f), pos_escritura); // recortar el archivo

```

Caso "expandir" o "repetir" (necesita 2 pasadas, backfill de atrás hacia adelante):

```

// Pasada 1: contar cuántos elementos se duplican
long cant = 0, repetidos = 0;
while (fread(&dato, sizeof(dato), 1, f) == 1) {
    cant++;
    if (condicion) repetidos++;
}
long tam_original = cant * sizeof(dato);
long tam_nuevo = (cant + repetidos) * sizeof(dato);

// Reservar espacio al final
fseek(f, tam_nuevo - 1, SEEK_SET);
fputc(0, f);

// Pasada 2: mover desde atrás hacia adelante
long pos_lect = tam_original - sizeof(dato);
long pos_escr = tam_nuevo - sizeof(dato);
while (pos_lect >= 0) {
    fseek(f, pos_lect, SEEK_SET);
    fread(&dato, sizeof(dato), 1, f);
    fseek(f, pos_escr, SEEK_SET);
    fwrite(&dato, sizeof(dato), 1, f);
    pos_escr -= sizeof(dato);
    if (condicion) {
        fseek(f, pos_escr, SEEK_SET);
        fwrite(&dato, sizeof(dato), 1, f);
        pos_escr -= sizeof(dato);
    }
    pos_lect -= sizeof(dato);
}
fclose(f);

```

7. SOCKETS (C++)

Template servidor (el más común en los finales):

```

int main(int argc, char *argv[]) {
    int puerto = std::atoi(argv[1]);
    // Si hay IP: char *ip = argv[1]; int puerto = std::atoi(argv[2]);

    int socket_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (socket_fd < 0) return -1;

    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY; // si no hay IP
    // addr.sin_addr.s_addr = inet_addr(ip); // si hay IP
    addr.sin_port = htons(puerto);

    if (bind(socket_fd, (struct sockaddr*)&addr, sizeof(addr)) < 0) return
-1;
    if (listen(socket_fd, 1) < 0) return -1;

    int client_fd = accept(socket_fd, nullptr, nullptr);
    if (client_fd < 0) return -1;

    // --- lógica del programa: recv / send ---
    char c;
    while (recv(client_fd, &c, 1, 0) > 0) {
        // procesar c
    }

    close(client_fd);
    close(socket_fd);
    return 0;
}

```

Template cliente:

```

if (connect(socket_fd, (struct sockaddr*)&addr, sizeof(addr)) < 0) return
-1;

```

Puntos clave:

- Recibir char a char con `recv(fd, &c, 1, 0)` es el patrón más común
- `send` / `recv` devuelven cantidad de bytes enviados/recibidos (≤ 0 = error/cierre)
- Siempre cerrar `client_fd` y `socket_fd` al final
- `htons()` convierte el puerto a network byte order (big-endian)

8. CLASES C++ — OPERADORES Y CONSTRUCTORES

Plantilla completa de clase con todo lo pedido:

```

class Punto {
private:
    float x, y;
public:
    Punto() : x(0), y(0) {}
    Punto(float a, float b) : x(a), y(b) {}

    // Constructor copia
    Punto(const Punto& o) : x(o.x), y(o.y) {}

    // Constructor move
    Punto(Punto&& o) noexcept : x(o.x), y(o.y) { o.x = 0; o.y = 0; }

    // Operadores
    Punto& operator++() { ++x; ++y; return *this; }           // pre-
incremento
    Punto operator++(int) { Punto t(*this); x++; y++; return t; } // post-
incremento

    Punto operator+(const Punto& o) const { return Punto(x+o.x, y+o.y); }
    Punto operator-(const Punto& o) const { return Punto(x-o.x, y-o.y); }
    bool operator==(const Punto& o) const { return x==o.x && y==o.y; }

    // Impresión y carga (friend para acceder a privados)
    friend std::ostream& operator<<(std::ostream& out, const Punto& p) {
        out << p.x << " " << p.y; return out;
    }
    friend std::istream& operator>>(std::istream& in, Punto& p) {
        in >> p.x >> p.y; return in;
    }

    // Casteo explícito
    explicit operator float() const { return std::sqrt(x*x + y*y); }

    ~Punto() = default;
};

```

Diferencia copia vs move:

- **Copia** (`const T& otro`): duplica, no modifica el original. Más costoso.
- **Move** (`T&& otro`): transfiere, deja el original vacío. Más eficiente.

Evitar copia/clonación:

```

Clase(const Clase&) = delete;
Clase& operator=(const Clase&) = delete;

```

9. TEMPLATES

```
template <typename T>
class Pila {
    T datos[100];
    int tope = -1;
public:
    void push(const T& v) { datos[++tope] = v; }
    T pop() { return datos[tope--]; }
};
```

¿Por qué la implementación va en el .h?

El compilador necesita ver la definición completa del template en el punto donde se instancia. Si solo ve la declaración en el .h y la definición está en el .cpp, no puede generar el código para cada tipo.

Especialización parcial:

```
template <typename A, typename B> struct Cosas { /* general */ };
template <typename A>                struct Cosas<A, int> { /* si B es int */ };
```

10. THREADS, MUTEX Y CONDITION VARIABLES

Thread básico:

```
void mi_funcion() { /* ... */ }
int main() {
    std::thread t(mi_funcion);
    t.join();    // esperar a que termine
}
```

Mutex:

```
std::mutex mtx;
int contador = 0;

void incrementar() {
    std::lock_guard<std::mutex> lock(mtx); // RAII: se libera al salir del
    scope
    contador++;
}
```

Condition Variable (productor-consumidor):

```
std::mutex mtx;
std::condition_variable cv;
bool listo = false;

void productor() {
    std::unique_lock<std::mutex> lock(mtx);
    listo = true;
    cv.notify_one(); // o notify_all() para despertar a todos
}

void consumidor() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, [] { return listo; }); // espera hasta que listo == true
    // usar el dato
}
```

Deadlock:

Ocurre cuando dos hilos esperan mutuamente el lock del otro. El hilo A tiene m1 y espera m2; el hilo B tiene m2 y espera m1. Ninguno avanza.

Conceptos clave:

- **Critical Section:** parte del código que no deben ejecutar dos hilos al mismo tiempo.
- **Recursos compartidos:** heap, variables globales, file descriptors.
- **Recursos exclusivos por hilo:** stack, registros, program counter.

11. RAII

"Resource Acquisition Is Initialization": el recurso se adquiere en el constructor y se libera en el destructor. La vida del recurso está ligada al objeto.

```
class Archivo {
    FILE* f;
public:
    Archivo(const char* nombre) { f = fopen(nombre, "r+"); }
    ~Archivo() { if (f) fclose(f); }
};
// Al salir del scope, ~Archivo() se llama automáticamente
```

`std::lock_guard`, `std::unique_ptr`, `std::shared_ptr` son todos ejemplos de RAII en la STL.

12. SMART POINTERS

Tipo	Cuándo usar
<code>std::unique_ptr<T></code>	Propiedad exclusiva. No permite copias, sí moves.
<code>std::shared_ptr<T></code>	Propiedad compartida. Usa conteo de referencias.
<code>std::weak_ptr<T></code>	Observar un objeto de <code>shared_ptr</code> sin aumentar el conteo.

```
auto p = std::make_unique<int>(42); // no necesita delete
```

13. HERENCIA Y POLIMORFISMO

Orden de construcción/destrucción:

- **Construcción:** primero la clase base (A), luego la derivada (B).
- **Dstrucción:** primero la derivada (B), luego la base (A).

Método virtual:

```
class Animal {
public:
    virtual void hablar() { std::cout << "Animal habla\n"; }
};
class Perro : public Animal {
public:
    void hablar() override { std::cout << "Perro ladra\n"; }
};
Animal* a = new Perro();
a->hablar(); // imprime "Perro ladra" (despacho dinámico)
```

Destructor virtual:

- **Cuándo usarlo:** siempre que la clase sea base de herencia y se use polimorfismo (se haga `delete` con puntero a la base).
- Sin destructor virtual, al hacer `delete` de un puntero a la clase base, el destructor de la derivada NO se llama → memory leak.

```
class Base { public: virtual ~Base() {} };
class Derivada : public Base { /* ... */ ~Derivada() override {} };
```

14. FUNCTORS

Clase con `operator()` sobrecargado. Mantiene estado entre llamadas.

```
class Multiplicador {
    int factor;
public:
    Multiplicador(int f) : factor(f) {}
    int operator()(int x) const { return x * factor; }
};
Multiplicador doble(2);
int resultado = doble(5); // 10
```

15. FRIEND

Permite que una función/clase externa acceda a miembros privados:

```
class DNI {
    long numero;
public:
    friend void imprimir(const DNI& d); // puede ver DNI::numero
};
void imprimir(const DNI& d) { std::cout << d.numero; }
```

16. ITERADORES STL

```
std::vector<int> v = {10, 20, 30};
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it;
}
// Forma moderna:
for (const auto& elem : v) { std::cout << elem; }
```

17. NAMESPACES

```
namespace A { void imprimir() { std::cout << "A\n"; } }
namespace B { void imprimir() { std::cout << "B\n"; } }
int main() {
    A::imprimir();
    B::imprimir();
}
```

18. ACCESIBILIDAD EN CLASES

Modificador	Acceso
<code>public</code>	Cualquier lugar
<code>protected</code>	Solo la clase y sus clases hijas
<code>private</code>	Solo dentro de la clase

19. FUNCIÓN BLOQUEANTE Y THREADS

Una función es bloqueante cuando detiene la ejecución del programa (ej: `while` esperando input). Solución: ejecutarla en un thread separado para que el resto del programa continúe.

20. TEMPLATE DE CLASE PILA/LISTA (para el final)

```
template <typename T>
class Pila {
private:
    T datos[100];
    int tope;
public:
    Pila() : tope(-1) {}
    Pila(const Pila& p) : tope(p.tope) { for(int i=0;i<=tope;i++)
datos[i]=p.datos[i]; }
    Pila(Pila&& p) noexcept : tope(p.tope) {
        for(int i=0;i<=tope;i++) datos[i]=std::move(p.datos[i]);
        p.tope=-1;
    }
    void push(const T& v) { datos[++tope] = v; }
    T pop() { return datos[tope--]; }
    bool vacia() const { return tope == -1; }
    explicit operator bool() const { return !vacía(); }
    friend std::ostream& operator<<(std::ostream& out, const Pila& p) {
        for(int i=0;i<=p.tope;i++) out << p.datos[i] << " "; return out;
    }
};
```

RESUMEN DE TEMAS MÁS FRECUENTES EN FINALES

Según los ejercicios del notebook, los temas que más aparecen son:

1. **Archivos en C** (procesar sobre sí mismo, big-endian, base N) → aparece en TODOS los finales
2. **Sockets** → aparece en TODOS los finales
3. **Clase con operadores** (++, ==, <<, >>, move) → aparece en TODOS los finales
4. **Pregunta teórica de C++** (mutex, RAII, templates, virtual, etc.) → aparece en TODOS los finales
5. **Threads/mutex** → frecuente como código o teoría